Der Python-Code einfach erklärt

Diese Erklärung geht das Notebook `Vortrag4_Berkeley_Analyse.ipynb` Zelle für Zelle durch. Lange Codestellen sind mit `...` gekürzt, der vollständige Code steht im Notebook. Du musst nicht jedes Detail auswendig können. Wichtig ist, dass du bei jedem Schritt sagen kannst, **was** er macht und **warum** wir ihn brauchen. Dafür steht bei jedem Abschnitt ein Kasten **„In einem Satz“**, den du so in der Präsentation sagen kannst.

0. Das Programm in 1 Minute

Das Programm macht fünf Dinge, immer in dieser Reihenfolge:

Schritt	Was passiert	Ergebnis
1. Erfassen	Tabelle 4 abtippen und in Zahlen umwandeln	eine Tabelle mit Zahlen statt Text
2. Prüfen	Die Zeile «Total» nachrechnen	30.4 % und 44.6 %, die Daten stimmen
3. Anzahlen	Die echten Anzahlen (wie viele Personen) laden	1 835 Frauen, 2 691 Männer
4. Analysieren	Quoten vergleichen, Unabhängigkeit testen	abhängig gesamt, unabhängig je Fachbereich
5. Darstellen	Diagramme zeichnen	5 Bilder für Folien und Ausarbeitung

Wichtige Begriffe vorab:

Begriff	Was es ist	Beispiel
Variable	Ein Name, unter dem man einen Wert speichert	`n = 4526`
String	Text, steht in Anführungszeichen	`"82.4%"`
Liste	Mehrere Werte in eckigen Klammern	`["A", "B", "C"]`
Tupel	Wie eine Liste, aber in runden Klammern und unveränderlich	`(89, 19)`
Dictionary	Zuordnung Schlüssel → Wert, in geschweiften Klammern	`{"Frauen": 0.304}`
Funktion	Ein Stück Code mit Namen, das man aufrufen kann	`def gesamtquote(...)`
DataFrame	Eine Tabelle in pandas, wie ein Excel-Blatt	`tab4`
`#`	Alles nach `#` ist ein Kommentar, Python ignoriert es	`# Erklärung`

1. Imports: die Werkzeuge laden

```
import re
from functools import partial
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
```

- `import` lädt eine **Library**, also eine fertige Werkzeugkiste, die andere programmiert haben.
- `as np`, `as pd`, `as plt` sind **Abkürzungen**. Statt `pandas.DataFrame` schreiben wir `pd.DataFrame`.
- `from ... import ...` holt nur **ein bestimmtes Werkzeug** aus der Kiste.

Library	Wofür wir sie brauchen
`re`	Text bereinigen, zum Beispiel das `%` entfernen
`partial`	Eine Funktion mit festgelegtem Argument erzeugen (Currying)
`numpy`	Rechnen mit ganzen Zahlenreihen auf einmal
`pandas`	Tabellen (DataFrames)
`matplotlib`	Diagramme zeichnen
`chi2_contingency`	Der Chi-Quadrat-Test

```
FARBEN = {"Frauen": "#eb6834", "Männer": "#2a78d6"}
plt.rcParams.update({...})
ABB_ORDNER = "abbildungen/"
```

- `FARBEN` ist ein **Dictionary**: Frauen bekommen immer Orange, Männer immer Blau. So sehen alle Diagramme gleich aus.
- `plt.rcParams.update(...)` stellt das Aussehen aller Diagramme ein (Schriftgrösse, keine Rahmen oben und rechts).
- `ABB_ORDNER` ist der Ordner, in dem die Bilder gespeichert werden.
- Grossgeschriebene Namen wie `FARBEN` sind **Konstanten**: Sie werden nur gelesen, nie verändert.

In einem Satz: «Zuerst laden wir die Libraries: pandas für Tabellen, numpy zum Rechnen, matplotlib für Diagramme und scipy für den Chi-Quadrat-Test.»

2. Tabelle 4 erfassen und in Zahlen umwandeln

2.1 Die Tabelle abtippen

```
tabelle4_roh = {
    "Fachbereich":   ["A", "B", "C", "D", "E", "F"],
    "Quote_Frauen":  ["82.4%", "68.0%", "34.1%", "34.9%", "23.9%", "7.0%"],
    ...
}
TOTAL_QUOTE = {"Frauen": 0.304, "Männer": 0.445}
```

- Wir tippen Tabelle 4 **genau so ab, wie sie gedruckt ist**, also als Text mit `%`.
- Es ist ein **Dictionary**: Der Schlüssel ist der Spaltenname, der Wert ist eine **Liste** mit den 6 Werten der Spalte.
- `TOTAL_QUOTE` speichert die Zeile «Total» aus Tabelle 4, damit wir später vergleichen können.

Warum als Text? So sieht man beim Vergleichen mit dem Aufgabenblatt sofort, ob richtig abgetippt wurde. Die Umwandlung in Zahlen macht danach der Code.

2.2 Drei kleine Funktionen zum Bereinigen

```
def entferne_prozentzeichen(wert):
    return re.sub(r"[%\s]", "", wert)
```

- `def` erstellt eine **Funktion** mit dem Namen `entferne_prozentzeichen`.
- `wert` ist das **Argument**, also das, was man der Funktion gibt.
- `re.sub(muster, ersatz, text)` ersetzt alles, was zum Muster passt.
- Das Muster `[%\s]` heisst: «ein Prozentzeichen oder ein Leerzeichen».
- Der Ersatz `""` ist leer, also wird es **gelöscht**.
- `return` gibt das Ergebnis zurück: aus `"82.4%"` wird `"82.4"`.

```
def in_anteil(wert):
    try:
        return float(wert) / 100
    except ValueError:
        raise ValueError(f"Ungültiger Tabellenwert: {wert!r}")
```

- `float(wert)` wandelt Text in eine Kommazahl um: `"82.4"` → `82.4`.
- `/ 100` macht daraus einen Anteil: `82.4` → `0.824`.
- `try` heisst: «Versuch das mal.»
- `except ValueError` heisst: «Falls genau dieser Fehler passiert, mach Folgendes.»
- Ein `ValueError` entsteht, wenn der Text keine Zahl ist, zum Beispiel bei einem Tippfehler wie `"82,4"` mit Komma.

- Dann geben wir eine **verständliche Fehlermeldung** aus: «Ungültiger Tabellenwert: '82,4'».
- **Warum nur `ValueError`?** Andere Fehler deuten auf echte Programmierfehler hin. Die wollen wir sehen und nicht verstecken. Das empfiehlt auch McKinney.

2.3 Funktionen in einer Liste

```
bereinigungs_schritte = [str.strip, entferne_prozentzeichen, in_anteil]

def bereinige(werte, schritte):
    ergebnis = []
    for wert in werte:
        for funktion in schritte:
            wert = funktion(wert)
        ergebnis.append(wert)
    return ergebnis
```

- `bereinigungs_schritte` ist eine **Liste von Funktionen**. Das geht, weil Funktionen in Python **Objekte** sind (McKinney 3.2).
- `str.strip` ist eine eingebaute Funktion, die Leerzeichen am Anfang und Ende entfernt.
- `bereinige` hat **zwei Schleifen** (`for`):
- Die äussere geht durch jeden Wert der Spalte.
- Die innere wendet jede Funktion der Liste nacheinander auf diesen Wert an.
- `ergebnis.append(wert)` hängt den fertigen Wert an die Ergebnisliste an.

So läuft es für einen Wert ab:

Schritt	Funktion	Wert danach
Start	–	`"82.4%"`
1	`str.strip`	`"82.4%"` (keine Leerzeichen da)
2	`entferne_prozentzeichen`	`"82.4"`
3	`in_anteil`	`0.824`

Vorteil: Braucht es einen weiteren Bereinigungsschritt, fügt man ihn nur in die Liste ein. Die Funktion `bereinige` bleibt gleich.

2.4 Die Tabelle bauen

```
tab4 = pd.DataFrame({
    spalte: werte if spalte == "Fachbereich" else bereinige(werte,
bereinigungs_schritte)
    for spalte, werte in tabelle4_roh.items()
}).set_index("Fachbereich")
```

Das sieht kompliziert aus, macht aber nur das:

- Gehe durch jede Spalte von `tabelle4_roh`.
- Ist es die Spalte «Fachbereich», lass sie so (A, B, C …).
- Bei allen anderen Spalten: wende `bereinige` an, mache also aus Text Zahlen.
- `pd.DataFrame(...)` macht daraus eine **Tabelle**.
- `.set_index("Fachbereich")` macht die Buchstaben A–F zur **Zeilenbeschriftung**.

Ergebnis:

Fachbereich	Quote_Frauen	Quote_Männer	Anteil_Frauen	Anteil_Männer
A	0.824	0.621	0.059	0.307
B	0.680	0.630	0.014	0.208
…	…	…	…	…

In einem Satz: «Wir tippen Tabelle 4 als Text ab und wandeln sie mit einer Liste von Funktionen in Zahlen um. Bei einem Tippfehler bekommen wir eine klare Fehlermeldung.»

3. Die Daten prüfen: Satz der totalen Wahrscheinlichkeit

```
def gesamtquote(quoten, anteile):
    return float((quoten * anteile).sum())

for g in ["Frauen", "Männer"]:
    berechnet = gesamtquote(tab4[f"Quote_{g}"], tab4[f"Anteil_{g}"])
    print(f"{g:7s}: berechnet {berechnet:.1%}   laut Tabelle {TOTAL_QUOTE[g]:.1%}")
```

- `quoten * anteile` multipliziert **Zeile für Zeile**: Quote von A mal Anteil von A, Quote von B mal Anteil von B usw.
- `.sum()` zählt alle 6 Ergebnisse zusammen.
- Das ist genau die Formel **$P(Z \mid G) = \Sigma\, P(F_i \mid G) \cdot P(Z \mid F_i, G)$**.

Für Frauen gerechnet:

FB	Anteil	× Quote	= Beitrag
A	0.059	0.824	0.049
B	0.014	0.680	0.010
C	0.323	0.341	0.110
D	0.204	0.349	0.071
E	0.214	0.239	0.051
F	0.186	0.070	0.013
Summe			**0.304 = 30.4 %**

- `for g in ["Frauen", "Männer"]` macht die Rechnung einmal für Frauen und einmal für Männer.
- `tab4[f"Quote_{g}"]` holt eine Spalte. Das `f` vor dem Text heisst **f-String**: `{g}` wird durch «Frauen» bzw. «Männer» ersetzt, also `"Quote_Frauen"`.
- `{berechnet:.1%}` zeigt die Zahl als Prozent mit einer Nachkommastelle: `0.304` → `30.4%`.

Ausgabe: Frauen 30.4 % (Tabelle 30.4 %), Männer 44.6 % (Tabelle 44.5 %). Der Unterschied von 0.1 kommt nur vom Runden in Tabelle 4.

> **In einem Satz:** «Mit dem Satz der totalen Wahrscheinlichkeit rechnen wir die Zeile Total nach. Sie stimmt bis auf Rundung, also sind die Daten richtig erfasst.»

4. Die echten Anzahlen laden

Warum? Tabelle 4 hat nur Prozente. Für den Chi-Quadrat-Test braucht man aber **Anzahlen**, also wie viele Personen.

```
ORIGINAL = {
    "Frauen": {"A": (89, 19), "B": (17, 8), ...},
    "Männer": {"A": (512, 313), ...},
}
```

- Ein **Dictionary in einem Dictionary**: zuerst das Geschlecht, dann der Fachbereich.
- Jeder Wert ist ein **Tupel** `(zugelassen, abgelehnt)`. Zum Beispiel Frauen in A: 89 zugelassen, 19 abgelehnt.
- Die Zahlen stammen aus der Originalstudie von Bickel et al. (1975).

```
def kennzahlen(zugelassen, abgelehnt, total_bewerbungen):
    bewerbungen = zugelassen + abgelehnt
    return zugelassen / bewerbungen, bewerbungen / total_bewerbungen
```

- Diese Funktion gibt **zwei Werte** zurück, mit Komma getrennt. In Wahrheit ist das **ein Tupel**.
- Beispiel: `quote, anteil = kennzahlen(89, 19, 1835)`
- bewerbungen = 89 + 19 = 108
- quote = 89 / 108 = 0.824 → 82.4 % ✓ wie Tabelle 4
- anteil = 108 / 1835 = 0.059 → 5.9 % ✓ wie Tabelle 4
- `quote, anteil = ...` **entpackt** das Tupel in zwei Variablen.

```
zeilen = []
for g, fachbereiche in ORIGINAL.items():
    for fb_name, (zul, abg) in fachbereiche.items():
        zeilen.append({"Fachbereich": fb_name, "Geschlecht": g, "Entscheid":
"zugelassen", "Anzahl": zul})
        zeilen.append({"Fachbereich": fb_name, "Geschlecht": g, "Entscheid":
"abgelehnt", "Anzahl": abg})
daten = pd.DataFrame(zeilen)
```

- `.items()` gibt Schlüssel und Wert zusammen. Die Schleife geht durch Frauen/Männer und dann durch A–F.
- Für jede Kombination entstehen **zwei Zeilen**: eine für zugelassen, eine für abgelehnt.
- So entsteht ein **«langes» Format** mit 2 × 6 × 2 = **24 Zeilen**:

Fachbereich	Geschlecht	Entscheid	Anzahl
A	Frauen	zugelassen	89
A	Frauen	abgelehnt	19
B	Frauen	zugelassen	17
…	…	…	…

Warum lang? Aus dieser einen Tabelle kann man jede andere Tabelle bauen: alle zusammen, nur ein Fachbereich, nur Frauen usw.

Die Gegenprobe: Danach rechnet eine Schleife mit `kennzahlen` alle 24 Prozentwerte aus und vergleicht sie mit Tabelle 4. `abs(quote - tab4...) > 0.0005` heisst: «Weicht der Wert um mehr als 0.05 Prozentpunkte ab?» (das ist die Rundungsgenauigkeit). **Ergebnis: 0 Abweichungen.** Die Anzahlen passen also perfekt zu Tabelle 4.

```
daten.pivot_table(index="Fachbereich", columns=["Geschlecht", "Entscheid"],
values="Anzahl", aggfunc="sum")
```

- `pivot_table` ist wie eine **Pivot-Tabelle in Excel**:
- `index` = was in die Zeilen kommt (Fachbereich),
- `columns` = was in die Spalten kommt (Geschlecht und Entscheid),
- `values` = welche Zahl drinsteht (Anzahl),
- `aggfunc="sum"` = zusammenzählen.

In einem Satz: «Weil Tabelle 4 nur Prozente hat, laden wir die Anzahlen aus der Originalstudie und prüfen per Code, dass sie Tabelle 4 exakt reproduzieren.»

5. Die Analyse-Funktionen (Herzstück)

5.1 `vierfeldertafel`: die 2×2-Tabelle bauen

```
def vierfeldertafel(daten, fachbereich=None):
    if fachbereich is not None:
        daten = daten[daten["Fachbereich"] == fachbereich]
    return daten.pivot_table(index="Geschlecht", columns="Entscheid",
                             values="Anzahl", aggfunc="sum")[["zugelassen",
"abgelehnt"]]
```

- `fachbereich=None` ist ein **Keyword-Argument mit Standardwert**.
- `vierfeldertafel(daten)` → nichts angegeben → `None` → **alle Fachbereiche** zusammen.
- `vierfeldertafel(daten, fachbereich="A")` → **nur Fachbereich A**.
- `daten[daten["Fachbereich"] == fachbereich]` ist ein **Filter**: Behalte nur die Zeilen, in denen der Fachbereich stimmt.
- `pivot_table` baut daraus die Vierfeldertafel.
- `[["zugelassen", "abgelehnt"]]` legt die Reihenfolge der Spalten fest.

Ergebnis für alle zusammen:

	zugelassen	abgelehnt
Frauen	557	1 278
Männer	1 198	1 493

5.2 `pruefe_unabhaengigkeit`: der wichtigste Teil

```
def pruefe_unabhaengigkeit(tafel, alpha=0.05):
    n = tafel.values.sum()
    p_w = tafel.loc["Frauen"].sum() / n
    p_z = tafel["zugelassen"].sum() / n
    p_z_und_w = tafel.loc["Frauen", "zugelassen"] / n
    chi2, p_wert, _, _ = chi2_contingency(tafel.values, correction=False)
    return {
        "n": int(n),
        "P(Z)": p_z,
        "P(Z|W)": ...,
        "P(Z|M)": ...,
        "P(Z∩W)": p_z_und_w,
        "P(Z)·P(W)": p_z * p_w,
        "Chi2": chi2,
        "p-Wert": p_wert,
        "unabhängig?": "ja (H0 nicht verworfen)" if p_wert >= alpha else "nein (H0
verworfen)",
    }
```

Zeile für Zeile mit den Zahlen für **alle Fachbereiche zusammen**:

Code	Bedeutung	Rechnung	Ergebnis
`n = tafel.values.sum()`	Alle Bewerbungen	557 + 1 278 + 1 198 + 1 493	**4 526**
`p_w = tafel.loc["Frauen"].sum() / n`	P(W): Anteil Frauen	1 835 / 4 526	**0.4054**
`p_z = tafel["zugelassen"].sum() / n`	P(Z): Anteil Zugelassene	1 755 / 4 526	**0.3878**
`p_z_und_w = ... / n`	P(Z ∩ W): zugelassen **und** Frau	557 / 4 526	**0.1231**
`p_z * p_w`	Was bei Unabhängigkeit herauskommen müsste	0.3878 × 0.4054	**0.1572**

- `tafel.loc["Frauen"]` holt die **Zeile** «Frauen», `tafel["zugelassen"]` die **Spalte** «zugelassen».
- **Vergleich:** 0.1231 ≠ 0.1572 → Zulassung und Geschlecht sind **abhängig**.
- `chi2_contingency(...)` macht den **Chi-Quadrat-Test** und liefert vier Werte. Wir brauchen nur die ersten zwei:
- `chi2` = Teststatistik = **92.2**
- `p_wert` = **$7.8 \cdot 10^{-22}$**, also praktisch 0
- `_` heisst in Python «diesen Wert brauche ich nicht».
- `correction=False` schaltet eine Korrektur aus, damit das Ergebnis genau der Formel aus der Vorlesung entspricht.

- `alpha=0.05` ist das **Signifikanzniveau** (5 %), wieder ein Keyword-Argument mit Standardwert.
- `"ja" if p_wert >= alpha else "nein"` ist ein **Wenn-dann in einer Zeile**: Ist der p-Wert mindestens 5 %, gilt «unabhängig», sonst «abhängig».
- **Rückgabe als Dictionary:** Die Funktion liefert 9 Werte. Mit Namen wie `ergebnis["p-Wert"]` findet man sie leichter als mit Nummern.

In einem Satz: «Die Funktion vergleicht P(Z ∩ W) mit P(Z) · P(W) und macht zusätzlich den Chi-Quadrat-Test. Sind die Werte gleich, sind Zulassung und Geschlecht unabhängig.»

5.3 Generator, Lambda und partial

```
def ergebnisse_je_fachbereich(daten, fachbereiche):
    for fb in fachbereiche:
        yield fb, pruefe_unabhaengigkeit(vierfeldertafel(daten, fachbereich=fb))
```

- `yield` statt `return` macht die Funktion zu einem **Generator**.
- Unterschied: `return` gibt **einmal** etwas zurück und ist fertig. `yield` gibt **einen Wert nach dem anderen**, immer erst, wenn der nächste verlangt wird («Lazy Evaluation»).
- Hier: Er liefert für A, dann B, dann C … jeweils `(Fachbereich, Testergebnis)`.

```
prozent = lambda x: f"{x:.1%}"
dezimal = lambda x: f"{x:.4f}"
```

- Ein **Lambda** ist eine **Mini-Funktion ohne Namen**, in einer Zeile.
- `prozent(0.3035)` → `"30.4%"`, `dezimal(0.12306)` → `"0.1231"`.
- Gleichbedeutend mit: `python def prozent(x): return f"{x:.1%}"`

```
pruefe_streng = partial(pruefe_unabhaengigkeit, alpha=0.01)
```

- `partial` erzeugt eine **neue Funktion**, bei der ein Argument schon festgelegt ist. Das nennt man **Currying**.
- `pruefe_streng(tafel)` ist dasselbe wie `pruefe_unabhaengigkeit(tafel, alpha=0.01)`, also der Test mit **1 %** statt 5 %.

6. Aufgabe (b): Das erste Diagramm

```
def balken_beschriften(ax, balken, format_funktion=prozent):
    for b in balken:
        ax.annotate(format_funktion(b.get_height()), ...)
```

- Schreibt über jeden Balken seinen Wert, zum Beispiel «30.4%».

- `format_funktion=prozent` : Hier wird eine **Funktion als Argument** übergeben. So kann man bestimmen, wie die Zahl aussieht.

```
gesamt = pruefe_unabhaengigkeit(vierfeldertafel(daten))
quoten_gesamt = {"Frauen": gesamt["P(Z|W)"], "Männer": gesamt["P(Z|M)"]}

fig, ax = plt.subplots(figsize=(6, 4.2))
balken = ax.bar(list(quoten_gesamt), list(quoten_gesamt.values()), color=[...])
...
fig.savefig(ABB_ORDNER + "abb1_gesamtquote.png", dpi=200)
plt.show()
```

Jedes Diagramm im Notebook folgt **demselben Muster**:

Schritt	Code	Bedeutung
1	`fig, ax = plt.subplots(...)`	Leeres Bild (fig) mit Zeichenfläche (ax) erstellen
2	`ax.bar(...)`	Balken zeichnen (Namen, Höhen, Farben)
3	`ax.set_ylabel`, `ax.set_title` …	Beschriften
4	`fig.savefig(...)`	Als PNG speichern (für Folien und Ausarbeitung)
5	`plt.show()`	Im Notebook anzeigen

- Zuerst wird die Vierfeldertafel für **alle** gebildet, dann geprüft. Aus dem Ergebnis nehmen wir die zwei Quoten: Frauen **30.4 %**, Männer **44.5 %**.
- `ax.annotate(...)` zeichnet den Doppelpfeil und den Text «14.2 Prozentpunkte weniger für Frauen».

```
n_frauen = int(daten.loc[daten.Geschlecht == "Frauen", "Anzahl"].sum())
print(round(n_frauen * quoten_gesamt["Frauen"]))   # 557
print(round(n_frauen * quoten_gesamt["Männer"]))   # 817
```

- Zählt alle Frauen zusammen (1 835) und rechnet: 1 835 × 30.35 % = **557** zugelassen.
- Mit der Quote der Männer wären es 1 835 × 44.52 % = **817** gewesen.

In einem Satz: «Für Aufgabe (b) genügt ein Balkendiagramm mit zwei Balken: Männer 44.5 %, Frauen 30.4 %. Das sieht man sofort.»

7. Aufgabe (c): Vergleich je Fachbereich

7.1 Quoten je Fachbereich und Differenz

```
def gruppiertes_balkendiagramm(tab, spalten_praefix, titel, ylabel, dateiname,
ylim=None):
    x = np.arange(len(tab.index))
    ...
    for i, g in enumerate(["Frauen", "Männer"]):
        balken = ax.bar(x + (i - 0.5) * (breite + 0.02), tab[f"
{spalten_praefix}_{g}"], ...)
```

- Zeichnet für jeden Fachbereich **zwei Balken nebeneinander**: Frauen und Männer.
- `np.arange(6)` → `[0, 1, 2, 3, 4, 5]`, also eine Position pro Fachbereich.
- `x + (i - 0.5) * ...` verschiebt die Frauen-Balken etwas nach links und die Männer-Balken etwas nach rechts.
- `enumerate` gibt die Nummer mit: `i = 0` für Frauen, `i = 1` für Männer.
- `spalten_praefix` ist der Trick: Mit `"Quote"` zeichnet die Funktion die **Quoten**, mit `"Anteil"` die **Verteilung der Bewerbungen**. **Eine Funktion, zwei Diagramme**, kein doppelter Code.

```
diff = ((tab4["Quote_Frauen"] - tab4["Quote_Männer"]) * 100).round(1)
print(list(diff[diff > 0].index))   # ['A', 'B', 'D', 'F']
```

- Rechnet für jeden Fachbereich Frauen minus Männer, in Prozentpunkten: A **+20.3**, B +5.0, C −2.8, D +1.8, E −3.8, F +1.1.
- `diff[diff > 0]` filtert die positiven Werte → Frauen sind in **A, B, D, F** besser, also **4 von 6**.

7.2 Streudiagramm: Frauenanteil gegen Zulassungsquote

```
fb = daten.pivot_table(index="Fachbereich", columns="Entscheid", values="Anzahl",
aggfunc="sum")
fb["Bewerbungen"] = fb["zugelassen"] + fb["abgelehnt"]
fb["Quote_gesamt"] = fb["zugelassen"] / fb["Bewerbungen"]
fb["Frauenanteil"] = (daten[daten.Geschlecht == "Frauen"].groupby("Fachbereich")
["Anzahl"].sum() / fb["Bewerbungen"])
r = np.corrcoef(fb["Frauenanteil"], fb["Quote_gesamt"])[0, 1]
```

- Pro Fachbereich rechnen wir drei neue Spalten aus:
- **Bewerbungen**: zum Beispiel A: 601 + 332 = 933
- **Quote_gesamt**: A: 601 / 933 = 64.4 %
- **Frauenanteil**: A: 108 Frauen / 933 = 11.6 %
- `groupby("Fachbereich")` heisst: «Gruppiere nach Fachbereich und zähle pro Gruppe zusammen.»

- `np.corrcoef(...)` berechnet die **Korrelation**: **r = −0.79**. Das Minus heisst: Je mehr Frauen, desto tiefer die Quote.
- `ax.scatter(...)` zeichnet die Punkte, die Grösse entspricht der Anzahl Bewerbungen.

7.3 Standardisierung: Was wäre bei gleicher Fachbereichswahl?

```
gewichte_alle = fb["Bewerbungen"] / fb["Bewerbungen"].sum()
quote_g = zul_g / bew_g
anteil_g = bew_g / bew_g.sum()

vergleich = pd.DataFrame({
    "tatsächlich (eigene Fachbereichswahl)": {
        g: gesamtquote(quote_g[g], anteil_g[g]) for g in ["Frauen", "Männer"]},
    "standardisiert (gleiche Fachbereichswahl)": {
        g: gesamtquote(quote_g[g], gewichte_alle) for g in ["Frauen", "Männer"]},
})
```

- `gewichte_alle`: Wie verteilen sich **alle** 4 526 Bewerbungen auf die Fachbereiche? (A 20.6 %, B 12.9 %, …)
- `quote_g`: Quote je Fachbereich und Geschlecht, `anteil_g`: Verteilung je Geschlecht.
- **Der Trick:** Wir rufen **dieselbe Funktion** `gesamtquote` zweimal auf:
- **tatsächlich** mit den eigenen Gewichten jedes Geschlechts → Frauen 30.4 %, Männer 44.5 %
- **standardisiert** mit **denselben** Gewichten für beide → Frauen **43.0 %**, Männer **38.7 %**
- **Ergebnis:** Bei gleicher Fachbereichswahl würden Frauen sogar **häufiger** zugelassen. Der Unterschied kehrt sich um.

> **In einem Satz:** «Mit derselben Funktion gesamtquote, nur mit gleichen Gewichten für beide, sehen wir: Bei gleicher Fachbereichswahl läge die Quote der Frauen sogar höher.»

7.4 Stochastische Unabhängigkeit je Fachbereich

```
je_fb = pd.DataFrame(dict(ergebnisse_je_fachbereich(daten, tab4.index))).T
```

Von innen nach aussen gelesen:

1. `ergebnisse_je_fachbereich(...)`: Der **Generator** liefert für A–F je ein Paar `(Fachbereich, Ergebnis)`.
2. `dict(...)`: macht daraus ein Dictionary `{"A": {...}, "B": {...}, ...}`.
3. `pd.DataFrame(...)`: macht daraus eine Tabelle.
4. `.T` (transponieren): **dreht** die Tabelle, sodass die Fachbereiche in den Zeilen stehen.

Ergebnis: Nur **A** ist abhängig (zugunsten der Frauen), **B–F** sind unabhängig (alle p > 0.3).

```
for name in tab4.index:
    print(f"FB {name}:", pruefe_streng(vierfeldertafel(daten, fachbereich=name))
["unabhängig?"])
```

- Derselbe Test noch einmal mit **α = 1 %** (das ist `pruefe_streng` aus `partial`).
- Ergebnis bleibt gleich: nur A abhängig. Das Ergebnis ist also **robust**.

7.5 Woher kommt die Abhängigkeit?

```
for titel, tafel in [("Geschlecht vs. Fachbereich", geschlecht_x_fb),
                     ("Entscheid vs. Fachbereich", entscheid_x_fb)]:
    chi2, p, dof, _ = chi2_contingency(tafel.values)
```

- Zwei weitere Chi-Quadrat-Tests mit 6 × 2-Tabellen:
- **Geschlecht und Fachbereich** hängen stark zusammen: χ^2 = **1 068.4**, df = 5
- **Zulassung und Fachbereich** hängen stark zusammen: χ^2 = **778.9**, df = 5
- `dof` = degrees of freedom = **Freiheitsgrade** = (6 − 1) × (2 − 1) = 5.
- **Bedeutung:** Der Fachbereich hängt mit **beiden** zusammen. Er ist die **Drittvariable (Confounder)**, die den Gesamtunterschied erzeugt. Das ist das **Simpson-Paradoxon**.

8. Spickzettel: Welches Konzept aus McKinney 3.2 steckt wo?

Konzept (McKinney 3.2)	Wo im Code	Kurz erklärt
`def` und `return`	alle Funktionen	Funktion erstellen, Ergebnis zurückgeben
Positions- und Keyword-Argumente	`vierfeldertafel(daten, fachbereich=None)`	Keyword hat einen Standardwert
Namensraum / kein `global`	alle Funktionen	Daten kommen als Argument rein, Ergebnis per `return` raus
Mehrere Rückgabewerte (Tupel)	`kennzahlen`	`quote, anteil = kennzahlen(89, 19, 1835)`
Rückgabe als Dictionary	`pruefe_unabhaengigkeit`	9 Werte mit Namen
Funktionen sind Objekte	`bereinigungs_schritte`	Funktionen in einer Liste
Funktion als Argument	`balken_beschriften(..., format_funktion=prozent)`	Formatierung wird übergeben
Lambda	`prozent`, `dezimal`, Achsenbeschriftung	Mini-Funktion in einer Zeile
Currying / `partial`	`pruefe_streng`	Test mit festem α = 1 %
Generator / `yield`	`ergebnisse_je_fachbereich`	Ergebnisse einzeln auf Anfrage
Fehlerbehandlung	`in_anteil`	`try/except ValueError`

9. Wenn die Lehrperson fragt …

«Warum habt ihr so viele Funktionen?» → «Weil wir dieselben Schritte siebenmal brauchen: einmal für alle und sechsmal für jeden Fachbereich. Mit Funktionen schreiben wir den Code nur einmal.»

«Was macht `pivot_table`?» → «Das ist wie eine Pivot-Tabelle in Excel. Wir sagen, was in die Zeilen, was in die Spalten und welche Zahl hinein soll.»

«Was ist der Unterschied zwischen `return` und `yield`?» → «`return` gibt einmal etwas zurück und beendet die Funktion. `yield` liefert einen Wert nach dem anderen, erst wenn er gebraucht wird.»

«Warum `correction=False`?» → «Damit das Ergebnis genau der Formel aus der Vorlesung entspricht. Mit Korrektur wäre $\chi^2 = 91.6$ statt 92.2, am Ergebnis ändert das nichts.»

«Woher habt ihr die Anzahlen?» → «Aus der Originalstudie von Bickel et al. 1975. Unser Code prüft, dass sie alle Prozentwerte aus Tabelle 4 reproduzieren, mit null Abweichungen.»

«Was passiert bei einem Tippfehler in der Tabelle?» → «`float()` kann den Text nicht umwandeln und wirft einen `ValueError`. Den fangen wir ab und geben eine klare Fehlermeldung mit dem falschen Wert aus.»